

Padrões de Projeto (Design Patterns)

Do conceito de padrão de projeto à aplicação prática em PHP — com os catálogos Criacional, Estrutural e Comportamental, e exemplos reais em Laravel.

Trilha sequencial: cada padrão assume que o anterior já foi compreendido.

O que é um Design Pattern?

Um **Design Pattern** não é código pronto — é uma solução geral e reutilizável para um problema recorrente de design de software.



Solução testada

Já foi validada por milhares de desenvolvedores em problemas parecidos.



Vocabulário comum

Permite dizer "use um Observer aqui" em vez de explicar tudo de novo.



Reutilizável

É uma ideia, não uma biblioteca — se adapta à linguagem e ao contexto.

Por que usar Design Patterns?

Eles existem porque **desenvolvedores resolvem os mesmos problemas repetidamente** — programar sem eles é reinventar soluções já testadas.



Manutenção mais fácil

Mudanças ficam isoladas em um único lugar, não espalhadas pelo sistema.



Código escalável

Novas regras entram sem reescrever o que já funciona.



Comunicação entre times

Facilita revisão de código e integração de novos desenvolvedores.

Categorias de Padrões de Projeto

Todo padrão de projeto pertence a uma destas famílias, de acordo com o problema que resolve.



Criacionais

Como objetos são criados. Ex: Singleton, Factory Method, Builder.



Estruturais

Como classes e objetos se compõem. Ex: Adapter, Decorator, Facade.



Comportamentais

Como objetos se comunicam e dividem responsabilidades. Ex: Strategy, Observer.

Singleton — Uma Única Instância

Garante que uma classe tenha apenas uma instância em toda a aplicação e fornece um ponto global de acesso a ela.

Quando usar

- Conexão única com banco de dados, compartilhada por toda a aplicação
- Configuração global lida uma única vez (ex: variáveis de ambiente)
- Log ou cache centralizado, evitando múltiplas instâncias concorrentes

Singleton em PHP

```
Conexao.php

class Conexao {
    private static ?Conexao $instancia = null;
    private function __construct() {}

    public static function get(): Conexao {
        if (self::$instancia === null) {
            self::$instancia = new self();
        }
        return self::$instancia;
    }
}
```



O construtor é private — ninguém pode criar uma instância com `new Conexao()` de fora da classe.

Singleton no Laravel

```
AppServiceProvider.php

public function register(): void
{
    $this->app->singleton(PagamentoGateway::class, function () {
        return new PagamentoGateway(config('services.stripe.key'));
    });
}
```

✓ O Service Container do Laravel já implementa Singleton para você — sem escrever a classe manualmente.

Factory Method — Criando sem Especificar a Classe

Define uma interface para criar um objeto, mas deixa uma classe especializada decidir qual subclasse instanciar.

Quando usar

- Sempre que existirem vários tipos de um mesmo objeto (ex: notificações por Email, SMS ou Push)
- Quando o código que usa o objeto não deveria conhecer sua classe concreta

Factory Method em PHP

```
NotificacaoFactory.php

interface Notificacao { public function enviar(string $msg): void; }

class EmailNotificacao implements Notificacao { /* ... */ }
class SmsNotificacao implements Notificacao { /* ... */ }

class NotificacaoFactory {
    public static function criar(string $tipo): Notificacao {
        return match ($tipo) {
            'email' => new EmailNotificacao(),
            'sms'   => new SmsNotificacao(),
        };
    }
}
```

Builder — Construindo Passo a Passo

Separa a construção de um objeto complexo da sua representação final, permitindo montá-lo em etapas.

Quando usar

- Objetos com muitos parâmetros opcionais (ex: montar uma consulta, um PDF, um e-mail)
- Quando um construtor com 8 parâmetros ficaria confuso e difícil de ler

Builder em PHP

```
EmailBuilder.php

class EmailBuilder {
    private array $partes = [];

    public function para(string $e): static { $this->partes['to'] = $e; return $this; }
    public function assunto(string $s): static { $this->partes['subject'] = $s; return $this; }
    public function build(): array { return $this->partes; }
}

$email = (new EmailBuilder())->para('a@x.com')->assunto('Oi')->build();
```



Cada método retorna `$this` — esse encadeamento é chamado de method chaining (interface fluente).

Adapter — Compatibilizando Interfaces

Permite que classes com interfaces incompatíveis trabalhem juntas, "traduzindo" uma chamada para outra.

Quando usar

- Integrar uma biblioteca externa ou legada com uma interface diferente da esperada
- Trocar de fornecedor (ex: gateway de pagamento) sem alterar o resto do sistema

Adapter em PHP

```
AdapterGatewayAntigo.php

// Biblioteca externa com interface diferente
class GatewayAntigo { public function pagar(float $valorEmCentavos) {} }

interface Pagamento { public function processar(float $valor): void; }

class AdapterGatewayAntigo implements Pagamento {
    public function __construct(private GatewayAntigo $gateway) {}
    public function processar(float $valor): void {
        $this->gateway->pagar($valor * 100);
    }
}
```

Decorator — Adicionando Comportamento

Adiciona novas responsabilidades a um objeto dinamicamente, sem alterar sua classe original.

Quando usar

- Combinar funcionalidades opcionais sem criar uma subclasse para cada combinação
- Adicionar comportamento em tempo de execução, não em tempo de compilação

Decorator em PHP

```
ComLeite.php

interface Cafe { public function preco(): float; }

class CafeSimples implements Cafe {
    public function preco(): float { return 5.0; }
}

class ComLeite implements Cafe {
    public function __construct(private Cafe $cafe) {}
    public function preco(): float { return $this->cafe->preco() + 2.0; }
}

$pedido = new ComLeite(new CafeSimples()); // 7.0
```

Facade — Simplificando um Subsistema

Fornece uma interface única e simples para um conjunto de classes complexas por trás dela.

Quando usar

- Um subsistema tem muitas classes e o consumidor só precisa de uma ação simples
- Você quer esconder a complexidade interna sem impedir o acesso direto quando necessário

Facade no Laravel

```
NotificacaoController.php

Storage::put('arquivo.txt', 'conteúdo');
Mail::to($user)->send(new BoasVindas());
Cache::remember('produtos', 60, fn () => Produto::all());
```

- ✓ Storage, Mail e Cache são Facades — por trás, chamam classes complexas registradas no Service Container.

Strategy — Trocando o Algoritmo em Tempo de Execução

Define uma família de algoritmos intercambiáveis e permite escolher qual usar sem alterar quem os utiliza.

Quando usar

- Cálculo de frete, formas de pagamento, regras de desconto que mudam com o contexto
- Quando um if/else gigante escolhe o comportamento com base em um tipo

Strategy em PHP

```
Carrinho.php

interface Desconto { public function aplicar(float $valor): float; }

class DescontoNatal implements Desconto {
    public function aplicar(float $valor): float { return $valor * 0.8; }
}

class SemDesconto implements Desconto {
    public function aplicar(float $valor): float { return $valor; }
}

class Carrinho {
    public function __construct(private Desconto $estrategia) {}
    public function total(float $v): float { return $this->estrategia->aplicar($v); }
}
```

Observer — Notificando Mudanças

Define uma dependência um-para-muitos: quando um objeto muda de estado, todos os observadores são avisados.

Quando usar

- Eventos do sistema — pedido pago, usuário cadastrado, estoque baixo
- Quando várias partes do sistema precisam reagir à mesma mudança, sem se conhecerem

Observer no Laravel

```
EquipamentoObserver.php

class EquipamentoObserver {
    public function created(Equipamento $equipamento): void {
        Log::info("Novo equipamento: {$equipamento->nome}");
    }
}

// AppServiceProvider.php
Equipamento::observe(EquipamentoObserver::class);
```

✓ O Eloquent já implementa Observer nativamente através dos Model Events.

Template Method — Esqueleto do Algoritmo

Define o esqueleto de um algoritmo na classe base e deixa que as subclasses sobrescrevam apenas certas etapas.

RelatorioBase.php

```
abstract class RelatorioBase {  
    public function gerar(): string {  
        return $this->cabecalho() . $this->corpo() . $this->rodape();  
    }  
    protected function cabecalho(): string { return "=== Relatório ==="; }  
    abstract protected function corpo(): string;  
}
```

Repository Pattern no Laravel

Isola a lógica de acesso a dados dos Controllers, escondendo se a origem é Eloquent, uma API externa ou cache.

```
EquipamentoRepository.php

interface EquipamentoRepository { public function todos(): Collection; }

class EquipamentoEloquentRepository implements EquipamentoRepository {
    public function todos(): Collection { return Equipamento::all(); }
}

class EquipamentoController {
    public function __construct(private EquipamentoRepository $repo) {}
    public function index() { return $this->repo->todos(); }
}
```

Quando (Não) Usar Padrões de Projeto

✓ Use quando...

- O problema é realmente recorrente, não hipotético
- O código vai crescer ou mudar de regras com frequência
- A equipe se beneficia de um vocabulário comum

⚠ Evite quando...

- A aplicação é simples e pequena
- O padrão adiciona complexidade sem benefício real
- Você o aplicaria "porque é boa prática", sem necessidade concreta (over-engineering)

⚠ Padrão errado no lugar errado é só complexidade a mais. Resolva o problema primeiro, nomeie o padrão depois.

Checklist: Qual Padrão Usar?

Padrão	Categoria	Use quando...
Singleton	Criacional	Precisar de uma única instância compartilhada
Factory Method	Criacional	Existirem vários tipos de um mesmo objeto
Builder	Criacional	O objeto tiver muitos parâmetros opcionais
Adapter	Estrutural	Integrar uma biblioteca com interface diferente
Decorator	Estrutural	Precisar adicionar comportamento sem herança
Facade	Estrutural	Quiser simplificar o acesso a um subsistema complexo
Strategy	Comportamental	Tiver algoritmos intercambiáveis
Observer	Comportamental	Precisar notificar várias partes de uma mudança